



# OMERTA AGENT

## Features & Full Usage Guide

An offline-capable coding & firmware agent that asks before it acts.

Runs on Android (no Termux), Linux, Kali, macOS, Windows, and any browser on your LAN.

Swappable brains: local models offline, frontier models online.

**Version 1.2.0**

always-ask · offline+online · unlimited chats · self-contained APK

# What OMERTA is

A complete agent framework that drives real backends. It is not a clone of Codex or Claude — model weights can't be copied out of the labs that made them — but it **calls** them (and any local model), with one hard rule: it proposes every action and runs nothing until you approve it. That rule is enforced in control flow, not in a prompt, and is covered by tests.

The Android app is fully self-contained: it embeds Python and runs the agent in-process, so there is no Termux and nothing external to set up. The backend is a normal HTTP service — no websocket, no dependency to install — and the same code runs on every platform.

## Feature matrix

| Capability                    | What you get                                                                                                                             |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Offline + online              | Local GGUF models (Ollama / llama.cpp / LM Studio) and 6 cloud providers, auto-switching, with an explicit Offline / Online / Auto mode. |
| Unlimited chats               | No message cap. Long conversations are compacted into a bounded context window so they never hit a wall. Truly unlimited offline.        |
| Always ask first              | Every command, file write, plugin side effect and connector call is proposed and waits for your yes. Enforced in code.                   |
| Learns your choices           | Every approve / deny / edit is recorded and fed back; denials become standing preferences.                                               |
| Sandbox                       | Tiered execution, a hard-deny list that can't be approved, and automatic backups before edits.                                           |
| Network safety                | Per-install token auth, loopback-exempt, spoof-proof against forwarded-header tricks.                                                    |
| Self-contained mobile         | Android APK embeds CPython (Chaquopy); the agent runs on-device with no Termux.                                                          |
| Connectors / plugins / skills | MCP client, Python plugin loader, keyword-loaded skill playbooks.                                                                        |
| Shared memory                 | Peer (LAN) and shared-folder sync; denials and deletions propagate across devices.                                                       |

# Architecture

## One agent, many front-ends

All request logic lives in `core/api.py`, so there is exactly one approval gate and one wire protocol no matter how you reach the agent:

- **CLI** — `omerta` in a terminal.
- **FastAPI server** (`server.py`) — for desktop / LAN, reachable from any browser.
- **Stdlib server** (`core/httpd.py`) — zero third-party deps; what the Android app runs, and a drop-in fallback anywhere FastAPI isn't installed.
- **Electron** desktop shell and the responsive **web UI**.

## Embedded Android backend

Chaquopy bundles CPython 3.11 plus pure-Python deps (`requests`, `pyyaml`) into the APK. A foreground service starts the stdlib server on `127.0.0.1`; the WebView loads the same UI. The agent's Python is the repo's `core/` and `tools/`, staged into the APK at build time and extracted on first launch — so the approval gate and deny-list can't drift from desktop.

```
MainActivity → BackendService → Chaquopy Python → omerta_boot → omerta_android → core.httpd
(127.0.0.1) ■■■ WebView (loopback, no token)
```

# Install & run

## Android (no Termux)

Install the APK, open it, and pick a brain in ■ → **MODE / MODEL ACCESS** — paste a cloud API key (online) or point it at a local model (offline). Nothing else to install.

## pip — Linux / macOS / Windows / Termux

```
pip install omerta_agent-1.0.0-py3-none-any.whl[all]
omerta # interactive CLI
omerta serve # web UI on :8787
omerta doctor # what works, what's missing
omerta sync # share memory across devices
```

## Give it a brain

```
export ANTHROPIC_API_KEY=sk-ant-... # online
ollama serve && ollama pull qwen2.5-coder:7b # offline, unlimited
```

# The always-ask guarantee

Every side-effecting action is proposed, never executed, until you answer. What you approve is character-for-character what runs.

| Answer | What happens                                                      |
|--------|-------------------------------------------------------------------|
| RUN    | Executes, is logged, and is remembered as approved.               |
| EDIT   | Your rewrite runs instead — a strong signal; it learns your form. |
| REFUSE | Never retried; the agent proposes a different approach.           |

## Risk tiers & the hard-deny list

- **DENY** — patterns like `rm -rf /`, `fastboot flashall -w`, `mkfs` on a raw disk: refused outright, not runnable even if you try to approve.
- **HIGH\_RISK** — flashing, `dd`, `git push --force`, `rm -rf`: shown with a loud destructive warning.
- **NORMAL / LOW\_RISK** — everything else; read-only commands run freely so the agent can actually think.

It generalizes: denying `fastboot erase userdata` teaches it about `fastboot erase` broadly. `/prefs` shows what it thinks it knows.

# Offline / Online / Auto & unlimited chats

| Mode    | Behavior                                                                                        |
|---------|-------------------------------------------------------------------------------------------------|
| Auto    | Online if reachable, otherwise a local model — switches on its own if you lose signal mid-task. |
| Offline | Local models only. Never touches the network. True unlimited — your hardware, your rules.       |
| Online  | Cloud providers only.                                                                           |

Chats have no message cap. To keep an endless conversation from overflowing the model's context window, only the most recent turns are replayed; older turns are already distilled into long-term memory, so nothing durable is lost.

## Model providers

| Provider             | Kind          | Net     | Default model                |
|----------------------|---------------|---------|------------------------------|
| claude / claude-opus | Anthropic     | online  | claude-sonnet-4-6 / opus-4-1 |
| openai               | OpenAI        | online  | gpt-4o                       |
| openrouter           | OpenAI-compat | online  | anthropic/claude-sonnet-4    |
| groq                 | OpenAI-compat | online  | llama-3.3-70b-versatile      |
| gemini               | OpenAI-compat | online  | gemini-2.0-flash             |
| ollama               | Ollama        | offline | qwen2.5-coder:7b             |
| llamacpp             | llama.cpp     | offline | local GGUF                   |

| Provider | Kind              | Net     | Default model |
|----------|-------------------|---------|---------------|
| lmstudio | OpenAI-compatible | offline | local model   |

The Anthropic provider works without the SDK (pure requests), so Termux needs no Rust toolchain for pydantic-core.

## Extending it

### Skills

Playbooks in `skills/<name>/SKILL.md`, auto-loaded by keyword. Ships with eight: `android-build`, `firmware-flash`, `cross-compile`, `apk-analysis`, `termux-env`, `git-hygiene`, `debug-build-failure`, `release-packaging`.

### Plugins

Drop a Python file in `plugins/` exposing a `register()` that returns tools; ~20 lines. Any tool marked `side_effects=True` is routed through the same approval gate — plugins cannot bypass `always-ask`.

### Connectors (MCP)

Any MCP server, stdio or HTTP, configured in `connectors.yaml` (GitHub, filesystem, git, sqlite, Sentry, Cloudflare presets). Tools appear as `mcp.<server>.<tool>` and every call needs approval.

### Importing what you already have

```
/import ~/Downloads/conversations.json # claude.ai or ChatGPT export
/import ~/notes # a markdown directory
/import ~/src/omerta-beats # a repo — learns its structure
```

## One brain across every device

What your phone learns, your Kali box knows — with no cloud account in the middle. Denials travel too: refusing a command on one device stops it being proposed on the others.

```
export OMERTA_SYNC_DIR=~/.Syncthing/omerta # shared folder (works offline)
/sync # sync via the shared folder
/sync 192.168.1.42:8787 # straight to another device on the LAN
/sync status # peers, folders, last sync
```

Rows carry a uid so re-syncing is idempotent; facts merge last-write-wins; tombstoned deletions propagate and are never resurrected; a DENIED choice is never dropped in a conflict — safety signals only accumulate.

## Security & audit

- **Model can't act alone** — every side effect suspends for approval; verified by tests.
- **Hard-deny list** — destructive patterns can't be approved at all.
- **Token auth** on the network server; loopback exempt unless the request carries forwarding headers (spoof-proof).  
uvicorn runs with proxy headers off.
- **On-device secrets** — API keys stored 0600 in app-private storage, never leave the device; the write path is loopback-only and off by default on LAN servers.
- **Auto-backups** before any file edit; checkpoints are restorable.
- **Per-project locking** in the threaded server; 16 MiB request cap.

Full threat model, findings and accepted design choices: docs/AUDIT.md. Re-run the checks with `bash tests/run_all.sh` (approval, deny-list, auth spoofing on both servers, tool-parse, mode routing, unlimited-chat trim, multi-device sync).

## Beyond chat — the engineering toolkit

OMERTA is a workflow system, not just a chat box. These verbs work from the CLI and, where they change the world, still pass through the approval gate.

### Codebase intelligence

`omerta index` builds a real index — Python parsed with `ast` (classes, functions, methods + imports + a call graph), other languages via conservative regex. Then `search / symbol / deps / calls` answer “where is this defined / imported / called” instantly. The same power is available to the agent as the **code\_index** plugin tools.

### Engineering roles

Ten focus profiles steer the one agent under the same gate: `omerta plan` (Architect), `review` (Reviewer, read-only), `build / test / debug`, and `Firmware / Security / Docs / Researcher / Developer`. `omerta role <name>` sets a default. `omerta workflow` chains roles (`plan→review`) or runs a **team concurrently** (`workflow team`) — thread-isolated, read-only, aggregating independent findings.

### Sandbox & recovery

`omerta sandbox snapshot` copies a workspace before a risky change and `rollback` restores it (edited and deleted files). Opt-in isolation (`OMERTA_ISOLATE=1`) wraps commands with resource limits and, when `bubblewrap/firejail` is installed, denies the network and shadows key stores (`~/.ssh`, `~/.aws`, `~/.gnupg`).

### Learning

`omerta teach "rule"` stores a durable project rule; `memory show / rules / forget` manage it. Approvals/denials are already learned automatically.

### Packaging

Ship it as a wheel, an **OCI/Docker image** (`docker/Dockerfile`), a validated **.deb** (`packaging/build-deb.sh`), a thin `AppImage`, the self-contained `APK`, or `Electron` installers. `scripts/build_all.sh` builds whatever the host can.

## HTTP API reference

| Method & path        | Purpose                                                             |
|----------------------|---------------------------------------------------------------------|
| POST /api/chat       | One chat turn: {project, kind, text cmd note} → full result.        |
| GET /api/status      | Providers, active model, mode, memory, skills, plugins, connectors. |
| POST /api/model      | Set the active provider (or 'auto').                                |
| POST /api/mode       | Set network mode: auto   offline   online.                          |
| GET/POST /api/secret | Read which keys are set (booleans only) / set one (loopback-only).  |
| GET /api/memory      | Search long-term memory + learned preferences.                      |
| GET /api/history     | Recent command log.                                                 |

| Method & path                                          | Purpose                           |
|--------------------------------------------------------|-----------------------------------|
| GET /api/sync/pull, POST /api/sync/push                | Peer memory exchange.             |
| POST /api/sync/run                                     | Trigger a sync ({peer} or {dir}). |
| POST /api/plugins/reload,<br>/api/connectors/reconnect | Hot-reload.                       |

## CLI / one-shot

```

omerta # interactive
omerta -c "audit this repo" # one-shot
omerta --project omerta-beats
omerta serve | doctor | sync | version
omerta teach "rule" | memory show | rules | role <name>
omerta index | search "q" | symbol Name | deps | calls Name
omerta git status|diff|log|review | evidence add <conf> "claim"
omerta plan/review/build/test/debug "... " | sandbox snapshot|rollback
omerta workflow <pipeline> "task"
omerta firmware inspect <img> | analyze <dtb|dtbo|boot.img> | extract <img> | report <dir>

```

## Firmware & device bring-up

OMERTA treats firmware the way a careful engineer does: evidence first, no invented hardware values. Every field is tagged **CONFIRMED** > **LIKELY** > **INFERRED** > **UNKNOWN**; UNKNOWN is a valid answer, so it says “UNKNOWN — evidence required” rather than guess a GPIO, regulator or panel timing.

| Read-only tool (Level 0) | What it does                                                                                                                                                                                              |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| inspect_image(path)      | Identify a boot / vendor_boot / dtbo image or raw .dtb; report header version (v0–v4), sizes, cmdline, os_version, and dt-table entries.                                                                  |
| analyze_dtb(path)        | Decode a device tree with a pure-Python FDT parser (no dtc) — a raw .dtb, a dtbo/dt_table entry, or the DTB embedded in a boot.img — into model/SoC/CPU/display/touch/regulators, each with a confidence. |
| board_report(dir)        | Fuse an adb evidence dir (getprop/cpuinfo/meminfo/partitions/dmesg + optional .dtb) into a BOARD REPORT with per-field confidence.                                                                        |
| collect_evidence_plan()  | The ordered, read-only adb commands to gather evidence.                                                                                                                                                   |

Safety levels map onto the approval gate: L0 read runs freely; L1 build / L2 modify / L3 device I/O are proposed and approved; L4 destructive (flash/erase/format) needs explicit approval and the hard-deny list blocks the unrecoverable ones outright. `firmware/t509k/` is a discovery-only TCL T509K (MT6765) starter tree, and the **firmware-bringup** skill drives the STOCK → EXTRACT → ANALYZE → RECONSTRUCT → BUILD → BOOT-TEST loop, one subsystem at a time.

## Build & release

```

bash scripts/build_all.sh # wheel, standalone binary, Electron, APK
cd android-native && gradle assembleDebug # self-contained APK

```

Cross-platform limits are real: PyInstaller can't cross-compile, macOS signing is mac-only, and the APK needs the Android SDK + a host Python 3.8–3.13 (Chaquopy runs pip on the host). See `INSTALL.md` and `android-native/README.md`.

OMERTA AGENT v1.2.0 — offline-capable coding & firmware agent that asks before it acts. This guide is generated from the repository docs.